SOFTWARE ENGINNERING QUESTION & ANSWERS

Q1. Define Software Engineering. what are the software characteristics? What are the various categories of software?

Ans) **Definition of Software Engineering**

Software Engineering is the systematic, disciplined, and organized approach to developing, designing, testing, and maintaining software.

Characteristics of Software

Software has some unique features that make it different from hardware:

1. **Intangible (Not Physical)**
   Software cannot be touched or seen physically like hardware.
2. **Developed, Not Manufactured**
   Software is created through coding and design, not produced in factories.
3. **Does Not Wear Out**
   Unlike hardware, software doesn't get damaged with use.
   (But it may degrade due to bugs or updates.)
4. **Highly Complex**
   Software systems can be very complicated with many components and interactions.
5. **Easy to Modify**
   Software can be updated, improved, or fixed easily compared to hardware.
6. **Custom-built or Generic**
   Software can be specially developed or made for general use.

Categories of Software

Software is mainly divided into the following categories:

1. System Software

- Controls and manages computer hardware.
- Provides a platform for other software.
- Examples: Operating Systems like Windows, Linux.

---

2. Application Software

- Designed for users to perform specific tasks.
- Examples: MS Word, Excel, web browsers.

---

3. Embedded Software

- Installed in hardware devices.
- Works in real-time systems.
- Examples: Software in washing machines, cars.

---

4. Utility Software

- Helps to maintain and optimize the system.
- Examples: Antivirus, disk cleanup tools.

---

5. Web Applications

- Run on web browsers using the internet.
- Examples: Gmail, online shopping websites.

---

6. Mobile Applications

- Designed for smartphones and tablets.
- Examples: WhatsApp, Instagram.

Q2) Define term software & software engineering. Software does not wear out state whether this statement is true or false. Justify your answer.

Ans) **Software** is a collection of computer programs and related documents that provide desired features, functionalities, and performance.

Software Engineering is a discipline in which theories, methods, and tools are applied in a systematic and organized way to develop high-quality software products.

The statement **"Software does not wear out" is TRUE.**

Justification:
Software is developed using logical components and is not a physical system, so it does not suffer from wear and tear like hardware.
However, software may **deteriorate (corrupt or spoil)** over time due to errors, bugs, or continuous modifications.

Q3) Elaborate how software engineering is a layered technology?

Ans) **Software Engineering as a Layered Technology**

Software Engineering is considered a **layered technology** because it is built using different layers that work together to develop high-quality software.

Layers of Software Engineering

1. **Quality (Foundation Layer)**
    - It is the **base of software engineering**.
    - Focuses on developing **high-quality software**.
    - All activities are performed keeping quality in mind.

---

2. **Process Layer (Backbone)**
    - It is the **foundation for all development activities**.
    - Defines a framework of tasks and activities.
    - Ensures proper planning, communication, and control.

---

3. **Methods Layer**
    - Provides **technical procedures** for building software.
    - Includes:
        - Requirement analysis
        - Design
        - Coding
        - Testing

---

4. **Tools Layer**
    - Provides **automated support** to the process and methods.
    - Examples: IDEs, testing tools, project management tools.

Q4) What are the various umbrella activities applied throughout a software project?

Ans) **Umbrella Activities in Software Engineering**

Umbrella activities are the activities that are applied **throughout the entire software development process** to ensure quality and proper management.

Various Umbrella Activities:

1. **Software Project Tracking and Control**
    - Monitors project progress.
    - Compares actual work with the planned schedule.

---

2. **Risk Management**

- Identifies and analyzes possible risks.
- Helps in reducing the impact of uncertainties.

---

3. **Software Quality Assurance (SQA)**
   - Ensures that the software meets required quality standards.
   - Includes systematic checking of processes and products.

---

4. **Formal Technical Reviews**
   - Conducted by experts to detect errors.
   - Improves quality and suggests improvements.

---

5. **Measurement**
   - Measures software size, effort, cost, and performance.
   - Helps in better project estimation and control.

---

6. **Software Configuration Management (SCM)**
   - Manages changes in software.
   - Maintains different versions and updates.

---

7. **Reusability Management**
   - Identifies components that can be reused.
   - Reduces development time and cost.

---

8. **Work Product Preparation and Production**
   - Involves creation of documents, reports, and user manuals.

Q5) What is software process framework? Explain in detail.

Ans) **Software Process Framework**

A **software process framework** is a structured set of activities that are followed to develop a software product. It provides a **basic structure (framework)** for planning, developing, and maintaining software.

👉 It divides the entire development process into a **small number of framework activities** that are applicable to all projects, regardless of size and complexity.

Framework Activities

The software process framework consists of the following **main activities**:

1. Communication

- Interaction between customer and developer.
- Requirements are gathered and understood.

---

2. Planning

- Estimation of time, cost, and resources.
- Scheduling and tracking of project activities.

---

3. Modeling

- Includes **requirement analysis and design**.
- Prepares flowcharts, algorithms, and system design.

---

4. Construction

- Actual development phase.
- Includes:
  - Coding
  - Testing

---

5. Deployment

- Delivery of software to the customer.
- Includes maintenance, support, and feedback.

Task Sets in Framework

- A framework is made up of **task sets**, which include:
  - Small work tasks
  - Milestones
  - Deliverables
  - Quality assurance points

Umbrella Activities

- These activities are applied **throughout all framework activities**, such as:
    - Risk management
    - Quality assurance
    - Project management
    - Configuration management

Q6) What are the elements of waterfall model? State its merits and demerits.

Ans) **Waterfall Model**

The **Waterfall Model** is a linear sequential software development model in which the development process flows step by step from one phase to the next.

Elements (Phases) of Waterfall Model

1. **Requirement Gathering and Analysis**
    - Requirements are collected and understood.
    - All requirements are documented.

---

2. **Design**
    - System design is prepared.
    - Includes data structure, architecture, and algorithms.

---

3. **Coding (Implementation)**
    - Actual programming is done based on design.

---

4. **Testing**
    - Software is tested to find and fix errors.
    - Ensures correct functionality.

---

5. **Maintenance**
    - After deployment, errors are corrected.
    - System is updated and improved.

Merits (Advantages)

1. Simple and easy to understand and implement.
2. Suitable for **small projects** with clear requirements.
3. Each phase is completed before moving to the next.
4. Easy to manage due to its structured approach.

Demerits (Disadvantages)

1. Difficult to handle changes once development starts.
2. Requirements must be clearly defined at the beginning.
3. Working model is available only at the end.
4. Not suitable for **complex or large projects**.
5. Sequential nature causes delays if any phase gets stuck.

Q7) What are customer myths? Discuss the reality of these myths?

Ans) **Customer Myths in Software Engineering**

Customer myths are **misunderstandings or false beliefs** held by customers about software development. These myths create confusion and can negatively affect the project.

Customer Myths and Their Reality

Myth 1:

A general statement of objectives is enough to start development; details can be added later.

👉 **Reality:**

- Clear and detailed requirements are necessary from the beginning.
- Lack of proper requirements can lead to **wrong software development** and rework.

Myth 2:

Requirements will change continuously, but changes can be easily accommodated because software is flexible.

👉 **Reality:**

- Although software is flexible, frequent changes increase:
    - Cost

- Development time
- Complexity

- Uncontrolled changes can **disturb the entire project plan**.

Q8) List and explain practitioner's myth and its realities?

Ans) **Practitioner's Myths in Software Engineering**

Practitioner's myths are **false beliefs held by developers (programmers)** about software development. These myths can lead to poor quality software and project failure.

Practitioner's Myths and Their Realities

Myth 1:

Once the program is written and working, the job is done.

👉 **Reality:**

- Software development does not end with coding.
- Activities like **testing, maintenance, and documentation** are equally important.

Myth 2:

Until the program is running, there is no way to assess its quality.

👉 **Reality:**

- Quality can be checked at every stage using:
  - Reviews
  - Testing
  - Verification techniques
- Early detection of errors saves time and cost.

Myth 3:

The only deliverable of a successful project is the working program.

👉 **Reality:**

- A complete software product includes:
  - Documentation
  - User manuals

    - Reports
- These are essential for maintenance and future use.

Myth 4:

There is no need for documentation; it slows down development.

👉 **Reality:**

- Documentation is very important for:
    - Understanding the system
    - Maintenance
    - Future updates
- Lack of documentation creates confusion later.

Q9) Explain with neat diagram incremental model and state its advantages and disadvantages.?

Ans) **Incremental Process Model**

The **Incremental Model** is a software development model in which the system is developed and delivered in **small parts (increments)**. Each increment adds new functionality to the previous version.

👉 The first release is called the **core product**, which contains basic features. Later increments enhance the system step by step.

Phases of Incremental Model

Each increment goes through the following phases:

1. **Analysis**
2. **Design**
3. **Coding**
4. **Testing**

Neat Diagram – Refer Unit 1 pptx.
Advantages (Merits)

1. Early delivery of working software (core product).
2. Easy to manage and test small parts.
3. Risk is reduced in each increment.
4. Customer feedback can be taken after each release.
5. Suitable when requirements are partially known.

Disadvantages (Demerits)

1. Requires good planning and design.
2. Total cost may be higher than other models.
3. Not suitable for very small projects.
4. Integration of all increments may be complex.
5. Requires continuous customer involvement.

Q10) Explain RAD model with the help of diagram.

Ans) **RAD Model (Rapid Application Development)**

The **RAD Model** is a type of **incremental process model** in which software is developed in a **very short time** using rapid development techniques.

👉 It is used when requirements are **well understood** and the system can be built using **reusable components**.

👉 A fully functional system can be developed in **60 to 90 days** using RAD.

Phases of RAD Model

1. **Business Modeling**
   - Defines business functions and information flow.
2. **Data Modeling**
   - Identifies data objects and relationships.
3. **Process Modeling**
   - Describes how data is processed.
4. **Application Generation**
   - Actual development using reusable components.
5. **Testing and Turnover**
   - Testing and delivery of the system.

Diagram Refer unit 1 pptx

Key Features

- Uses **multiple teams working in parallel**
- Focus on **fast development**
- Emphasis on **component-based construction**

Advantages

1. Very fast development.
2. Early delivery of system.
3. Reusability of components reduces effort.
4. Customer feedback is incorporated quickly.

Disadvantages

1. Requires highly skilled developers.
2. Needs strong customer involvement.
3. Not suitable for large or complex systems.
4. Requires more resources and multiple teams.

Q11) Describe agile in detail.

Ans) **Agile in Software Engineering**

Agile means the ability to **move quickly and respond to changes effectively**. It is a software development approach that focuses on **flexibility, customer collaboration, and rapid delivery of software**.

Definition of Agile

Agile is a development methodology that promotes:

- **Incremental development**
- **Continuous feedback**
- **Fast and flexible response to changes**

Characteristics of Agile Process

1. Adaptable to technical and environmental changes
2. Development is **incremental and iterative**
3. Continuous customer involvement
4. Frequent delivery of working software
5. Regular evaluation and improvement

Agile Principles

1. Customer satisfaction through early delivery
2. Welcome changing requirements at any stage
3. Deliver working software frequently
4. Developers and customers work together
5. Focus on simplicity and quality
6. Continuous improvement and technical excellence

Agile Methodologies

- **Scrum** – Work divided into small sprints
- **Extreme Programming (XP)** – Focus on continuous development and testing
- **Kanban** – Visual project management technique
- **DSDM, FDD, ASD** – Other agile approaches

Advantages (Merits)

1. High customer satisfaction
2. Easy to handle changing requirements
3. Faster delivery of software
4. Better communication and teamwork
5. Continuous feedback improves quality

Disadvantages (Demerits)

1. Less documentation
2. Difficult to predict cost and time
3. Can go off track without proper control
4. Requires skilled and experienced team

Q12) What is agility? Explain about process model.

Ans) **Agility in Software Engineering**

Agility is the ability to **create and respond to change quickly and effectively** in a software development environment.

👉 It focuses on:

- Fast response to changing requirements
- Continuous customer interaction
- Rapid delivery of software

Key Features of Agility

1. Adaptable to technical and environmental changes
2. Incremental and iterative development
3. Continuous customer feedback
4. Quick delivery in short time periods
5. Focus on teamwork and communication

Process Model in Software Engineering

A **process model** is a structured approach that defines the **sequence of activities** required to develop software. It provides a roadmap for development.

Generic Process Model Activities

1. **Communication** – Gathering requirements from customer
2. **Planning** – Estimation, scheduling, and tracking
3. **Modeling** – Analysis and design of system
4. **Construction** – Coding and testing
5. **Deployment** – Delivery and maintenance of software

Types of Process Models

1. **Prescriptive Models (SDLC)**
   - Waterfall model
   - Incremental model
   - RAD model
2. **Evolutionary Models**
   - Prototyping
   - Spiral model
3. **Other Models**

- Concurrent development model
- Component-based model

Q13) What is XP methology for project management?

Ans) **XP Methodology (Extreme Programming)**

Extreme Programming (XP) is an **Agile software development methodology** that focuses on **continuous development, customer satisfaction, and quick delivery** of software.

👉 It is mainly used for **project management and development** where requirements change frequently.

Key Features of XP

1. **Short development cycles (iterations)**
2. **Frequent releases of software**
3. **Continuous customer involvement**
4. **Focus on simplicity and quality**
5. **Rapid feedback and improvement**

XP Practices

1. **Incremental Planning**
    - Planning is done step by step for small releases.
2. **Small Releases**
    - Software is released in small parts frequently.
3. **Simple Design**
    - Keep design simple and easy to understand.
4. **Test-First Development (TDD)**
    - Write test cases before writing code.
5. **Refactoring**
    - Improve code structure without changing functionality.
6. **Pair Programming**
    - Two developers work together on one system.
7. **Collective Ownership**
    - Any team member can modify the code.
8. **Continuous Integration**

- Code is integrated and tested regularly.

Advantages of XP

1. High-quality software
2. Early detection of errors
3. Better customer satisfaction
4. Flexibility to changes
5. Faster development

Disadvantages of XP

1. Requires experienced developers
2. Less documentation
3. Not suitable for large projects
4. Needs continuous customer involvement

Q14) Write a note on Kanban.

Ans) **Kanban in Software Engineering**

Kanban is an **Agile project management technique** used to **visualize and manage workflow** in software development. It helps teams track tasks and improve efficiency.

Key Features of Kanban

1. **Visual Workflow**
   - Work is displayed on a **Kanban board** using columns like:
     To Do → In Progress → Done
2. **Work in Progress (WIP) Limits**
   - Limits the number of tasks being worked on at a time.
3. **Continuous Delivery**
   - Tasks are completed and delivered continuously (no fixed iterations).
4. **Flexibility**
   - Easy to add, remove, or change tasks anytime.

Working of Kanban

- Tasks are written on cards.
- Cards move across stages of development.
- Team members pick tasks based on priority.
- Progress is visible to everyone.

Advantages of Kanban

1. Improves visibility of work
2. Increases productivity
3. Flexible and easy to use
4. Helps identify bottlenecks

Disadvantages of Kanban

1. No strict deadlines
2. Can be difficult to track long-term progress
3. Requires discipline to maintain workflow

Q15) Explain Scrum in detail.

Ans) **Scrum in Software Engineering**

Scrum is an **Agile process framework** used for developing **complex software systems**. It is **iterative and incremental**, focusing on delivering working software in small parts called *increments*.

Key Principles of Scrum

1. Work is divided into **small teams**
2. Project is broken into **small parts (sprints)**
3. Changes can be easily accommodated
4. Focus on delivering **working software increments**
5. Continuous testing and documentation

Scrum Activities

1. **Backlog**
    - List of all project requirements.
2. **Sprint**

- A short time period (iteration) to complete a set of tasks.

3. **Daily Meetings**
    - Short meetings to discuss progress and issues.
4. **Demo (Increment Delivery)**
    - Completed work is shown to the customer.

Roles in Scrum

1. **Scrum Master**
    - Manages the process and removes obstacles.
2. **Team Members**
    - Developers who build the product.

Advantages of Scrum

1. High transparency
2. Flexible to changes
3. Improves productivity
4. Faster delivery of software

Disadvantages of Scrum

1. Difficult to track decisions
2. Non-functional requirements may be ignored
3. Requires experienced team

Q16) Explain features of JIRA Tool

Ans) **Features of JIRA Tool**

JIRA is a project management and issue-tracking tool used in software development for **bug tracking, task management, and project planning**.

Key Features of JIRA

1. **Bug and Issue Tracking**
    - Tracks bugs, errors, and issues in the project.
    - Helps manage and resolve problems efficiently.

---

2. **Project Planning**
    - Supports planning of development activities and iterations.
    - Helps in organizing tasks and resources.

---

3. **Iteration (Sprint) Management**
    - Used in Agile to plan and manage **sprints**.
    - Tracks progress of tasks in each sprint.

---

4. **Reports and Analytics**
    - Provides reports like:
        - Burndown chart
        - Sprint report
        - Control chart
    - Helps monitor project progress.

---

5. **Dashboard**
    - Displays project status visually.
    - Shows tasks, progress, and performance metrics.

---

6. **Component Management**
    - Divides project into smaller components or modules.
    - Makes task handling easier.

---

7. **Customization**
    - Workflows, fields, and issue types can be customized as per project needs.

Q17) Explain with advantages and disadvantage of Prototype?

Ans) **Prototyping Model in Software Engineering**

The **Prototyping Model** is a software development approach in which a **working model (prototype)** of the system is built to understand requirements and get user feedback.

👉 It helps in situations where requirements are **not clearly defined**.

Prototyping Process

1. **Quick Design** – A rough design of the system is prepared.
2. **Build Prototype** – A basic working model is developed.
3. **User Evaluation** – Users test and give feedback.
4. **Refinement** – Prototype is improved based on feedback.
5. **Final Product** – Final system is developed after requirements are clear.

Advantages (Merits)

1. Helps in better understanding of requirements
2. Early user involvement and feedback
3. Reduces chances of errors
4. Improves customer satisfaction
5. Useful when requirements are unclear

Disadvantages (Demerits)

1. Customer may expect final product too early
2. Initial prototype may have poor quality
3. Developers may take shortcuts in design
4. Can increase cost and development time
5. Not suitable for very large systems

Q18) Explain with advantages and disadvantage of Spiral Model?

Ans) **Spiral Model in Software Engineering**

The **Spiral Model** is an **evolutionary process model** that combines **iterative development with systematic risk analysis**. In this model, software is developed in a series of **spirals (iterations)**.

👉 It is mainly used for **large and complex projects** where risk management is important.

Working of Spiral Model

- Development is divided into multiple cycles (spirals).
- Each spiral consists of activities such as:

    - Planning
    - Risk analysis
    - Engineering (design & coding)
    - Evaluation
- At every stage, risks are identified and resolved before moving forward.

Diagram refer Unit 1 pptx

Advantages (Merits)

1. Risk identification and management at every stage
2. Suitable for large and complex projects
3. Allows changes in requirements
4. Continuous customer feedback
5. Better quality software

Disadvantages (Demerits)

1. Complex and difficult to manage
2. Expensive due to risk analysis
3. Requires skilled professionals
4. Not suitable for small projects
5. Success depends on proper risk assessment

Q19) what are the key factors that are considered for agile development.

Ans) **Key Factors for Agile Development**

Agile development focuses on **flexibility, customer satisfaction, and quick delivery**. The following key factors are considered:

---

1. Customer Satisfaction

- Highest priority is to deliver **valuable software early and continuously**.

---

2. Welcome Changing Requirements

- Changes are accepted at any stage of development.

- Agile adapts easily to new requirements.

---

3. Frequent Delivery

- Software is delivered in **small increments** within short time periods.

---

4. Continuous Customer Collaboration

- Developers and customers work together regularly.
- Ensures better understanding of requirements.

---

5. Teamwork and Communication

- Strong interaction among team members.
- Face-to-face communication is preferred.

---

6. Simplicity

- Focus on simple design and avoiding unnecessary work.

---

7. Continuous Improvement

- Regular evaluation and improvement of the process and product.

---

8. Technical Excellence

- Maintain high coding standards and good design practices.

Q20) How does pair programming work?

Ans) Pair programming is a collaborative software development technique where two programmers work together at one workstation.

One assumes the role of the driver, actively typing code, while the other acts as the navigator, reviewing each line for strategic direction, potential issues, and improvements.
They frequently switch roles to maintain engagement and share knowledge effectively.
This method enhances code quality through immediate error detection and fosters communication, speeding up problem-solving and reducing knowledge silos within the team.